

Lab # 10

JUnit Testing

Original Code:-

```
public class JUnitTesting {
    public int square (int x) {

        return x*x;
    }
    public int countA(String
word)
    {
        int count = 0;
        for (int i=0; i
<word.length();i++)
        {

            if(word.charAt(i)=='a' ||
word.charAt(i)=='A')

                {count++;

                }

            }
        return count;
    }
}
```

Refactored Code:-

```
StringMathUtils.java X SquareTest.java
1 public class StringMathUtils {
2     // Renamed method for clarity
3     public static int calculateSquare(int number)
4         return number * number;
5     }
6
7     // Added null safety and improved loop
8     public static int countLetterA(String word) {
9         if (word == null) {
10             return 0;
11         }
12         int count = 0;
13         for (char ch : word.toCharArray()) {
14             if (ch == 'a' || ch == 'A') {
15                 count++;
16             }
17         }
18         return count;
19     }
20 }
```

Original Unit Test:-

```
class squareTest {

    @Test

    void test() {

        JUnitTesting obj=new
JUnitTesting();

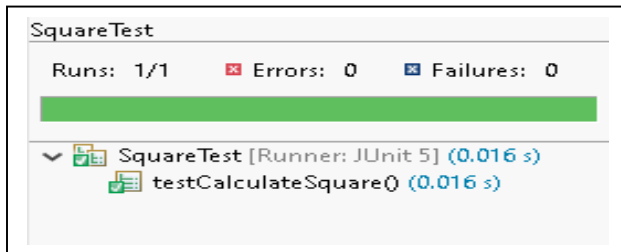
        int output=obj.square(5);

        assertEquals(25, output)

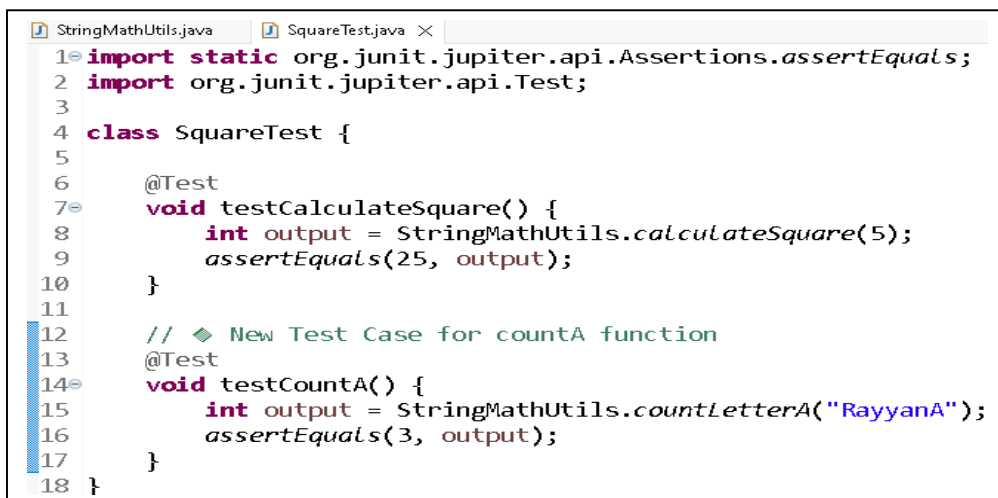
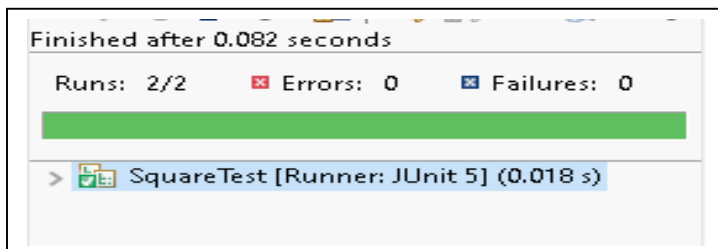
    }}
}
```

Updated Unit Test:-

```
StringMathUtils.java X SquareTest.java X
1 import static org.junit.jupiter.api.Assertions.assertEquals;
2 import org.junit.jupiter.api.Test;
3
4 class SquareTest {
5
6     @Test
7     void testCalculateSquare() {
8         int output = StringMathUtils.calculateSquare(5);
9         assertEquals(25, output);
10    }
11 }
```

Output:-**Lab Tasks:-**

- Add another test case for countA function (which is given in code).

Unit Test:-**Output:-**

- Make new project, make a class. Add 2 methods in it. One method will find the max integer present in the input integer array. The other method will find the min integer. Now create test cases for both these methods and test your code. Follow all the steps as mentioned above in the manual.

Code:-

```

1 public class ArrayUtils {
2     public int findMax(int[] numbers) {
3         if (numbers == null || numbers.length == 0) {
4             throw new IllegalArgumentException("Array cannot be empty");
5         }
6         int max = numbers[0];
7         for (int num : numbers) {
8             if (num > max) max = num;
9         }
10        return max;
11    }
12    public int findMin(int[] numbers) {
13        if (numbers == null || numbers.length == 0) {
14            throw new IllegalArgumentException("Array cannot be empty");
15        }
16        int min = numbers[0];
17        for (int num : numbers) {
18            if (num < min) min = num;
19        }
20        return min;
21    }
22 }

1 import static org.junit.jupiter.api.Assertions.*;
2 import org.junit.jupiter.api.Test;
3 import org.junit.jupiter.api.DisplayName;
4
5 public class ArrayUtilsTest {
6     private final ArrayUtils utils = new ArrayUtils();
7     @Test
8     @DisplayName("Should correctly find the maximum value")
9     void testFindMax() {
10         int[] data = {1, 9, 3, 7, 5};
11         int result = utils.findMax(data);
12         // Assert: Check if result is actually 9
13         assertEquals(9, result, "The max value should be 9");
14     }
15     @Test
16     @DisplayName("Should correctly find the minimum value")
17     void testFindMin() {
18         int[] data = {10, 2, 8, -5, 6};
19         int result = utils.findMin(data);
20         // Assert: Check if result is actually -5
21         assertEquals(-5, result, "The min value should be -5");
22     }
23     @Test
24     @DisplayName("Should throw exception for empty array")
25     void testEmptyArray() {
26         int[] empty = {};
27         assertThrows(IllegalArgumentException.class, () -> {
28             utils.findMax(empty);
29         }, "Empty array should trigger an exception");
30     }
31 }

```

Output:-

```

Finished after 0.088 seconds

Runs: 3/3      ✖ Errors: 0      ✖ Failures: 0

> [Icon] ArrayUtilsTest [Runner: JUnit 5] (0.021 s)

```